Practical Product Sampling Using Symbolic Warp Synthesis

HAOLIN LU* and XUJIE CHEN*, University of California, San Diego, USA

1 INTRODUCTION

Importance sampling is a core variance reduction technique when solving the Rendering Equation using Monte Carlo integration. However, it is hard to importance sample the full integrand:

$$\int_{\Omega} L_i(x, \omega_i) f_r(x, \omega_o, \omega_i) |\cos \theta_i| \mathbf{d}\omega_i. \tag{1}$$

When solving global illumination, the L_i term is a recursive integration. Therefore, path guiding [15] uses data-driven methods to approximate the term and achieve importance sampling roughly proportional to L_i. People also tried to take care of the $f_r \cdot |\cos \theta_i|$ term by approximate product sampling with some heuristics [3, 4], but there is no analytical way to combine both terms.

When solving direct illumination, the L_i term is defined by the known light sources, but product sampling does not become easier. For Next-Event Estimation (NEE), uniform solid angle sampling [1, 14] only considers the geometry term and radiance term, it would be helpful to do a cosine-weighted/projected solid angle sampling, which takes care of the foreshortening term as well. However, the explicit analytical routine for cosine-weighted solid angle sampling is only available for the spherical primitive [10, 12].

Meanwhile, the microfacet BRDF itself is usually a product of terms:

$$\rho(V, L) = \frac{F(V, H) D(H) G_2(V, L)}{4 \cos \theta_V \cos \theta_L}, \tag{2}$$

where VNDF sampling [6] only importance sample the D_V term related to D, but leaves some terms, $\frac{F \cdot G_2}{G_1}$, along. Combining NEE with full BRDF only gets more complicated.

Mathematicians have worked hard, but there is usually a last mile to go. Hart et al. [5] provides a very intriguing insight that we can do one or more extra warps on top of the existing sampling routine to walk down the final mile. We essentially need to fit a warp with the probability density function (PDF) proportional to the remaining terms, as composing warps ends up in multiplication for probability change:

$$p_w(\mathbf{x}') = p(\mathbf{x}) \left| \det \left(\frac{\partial w(\mathbf{x})}{\partial \mathbf{x}^R} \right) \right|^{-1} = p(\mathbf{x}) J_w(\mathbf{x}). \tag{3}$$

Hopefully, our warp only needs to deal with some very simple patterns, e.g. the cosine term in projected solid angle sampling. This behaviour motivates Hart et al. [5] to use simple warps like the bilinear and biquadratic ones to fit the sampling distributions.

While a bilinear patch might oversimplify the problem, normalizing flow goes to another extreme: it uses very heavy neural networks to approximate the sampling routine, which can be too heavy for real-time rendering. We can't help but wonder whether we could find something between them.

*Both authors contributed equally to this research.

Authors' Contact Information: Haolin Lu; Xujie Chen, University of California, San Diego, La Jolla, CA, USA.

2 MOTIVATION: SYNTHESISING SPHERICAL RECTANGLE SAMPLING

Spherical rectangle sampling [14] is derived with heavy handcraft: a clever coordinate reprojection, lots of manual integration and inversion, etc. However, the form is rather simple if we only observe the core computation that directly modifies the random variable u and v. As shown in fig. 1, it can be decomposed into a pipeline consisting of multiple locally invertible warps w_i, where the invertibility is shown in table 1. (Notice that we ignore the range and sign for now, which could be essential in practice, as many functions are invertible only within a specific range.)

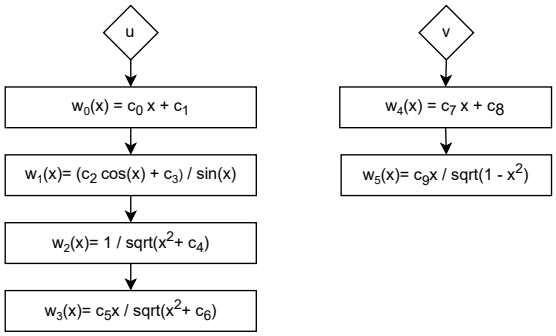

Fig. 1. The warp pipeline of spherical rectangle sampling [14].

Table 1. The warp functions used in fig. 1, and their inverse.

warp	inverse
$w_0(x) = c_0 \cdot x + c_1$	$w_0^{-1}(x) = \frac{x-c_1}{c_0}$
$w_1(x) = \frac{c_2\cos(x)+c_3}{\sin(x)}$	$w_1^{-1}(x) = 2\tan^{-1}\left(\frac{-\sqrt{c_2^2-c_3^2+x^2}-x}{c_2-c_3}\right)$
$w_2(x) = \frac{1}{\sqrt{x^2+c_4}}$	$w_2^{-1}(x) = \frac{\sqrt{1-c_4x^2}}{x}$
$w_3(x) = \frac{1}{\sqrt{x^2+c_4}}$	$w_3^{-1}(x) = \frac{\sqrt{c_6}x}{\sqrt{c_5^2-x^2}}$
$w_4(x) = c_7 \cdot x + c_8$	$w_4^{-1}(x) = \frac{x-c_8}{c_7}$
$w_5(x) = \frac{c_9x}{\sqrt{1-x^2}}$	$w_5^{-1} = \frac{x}{\sqrt{x^2+a^2}}$

The question is, could we synthesis this program from scratch? A typical warp-based sampling process requires a sampling routine $w(x)$, its inverse $w^{-1}(x)$, and the PDF evaluation $\mathbf{d}_x\left(w^{-1}(x)\right)$. The invertibility actually poses the main challenge during program synthesis. Janus [8] achieve invertibility by outputting extra information, which is not practical for sampling, as PDF computation will then need extra marginalization. Normalizing flow [11] achieve this by heavily constraining the operators to predefined coupling layers. However, most of the warps defined in table 1 have never been proposed as coupling layers in literature, although technically they could. This means the ground truth program will be out of the search space, even if we do a Neural Architecture Search (NAS) [13] of normalizing flows. In contrast, without human design, our approach could automatically enumerate potential warps (1D coupling layer) to enrich the search space.

3 SYMBOLIC WARP SYNTHESIS FOR 1D SAMPLING

We propose Symbolic Warp Synthesis, a novel pipeline to synthesis and fitting warps for sampling, as depicted in fig. 2. Before any actual training, we first enumerate as many basic symbolic warps w_i as possible, and we collect all the available warps in a list of primitives, described in section 3.1. At this stage, hopefully, we can include all the used primitive warps $w_0, w_1, \cdots, w_5$ used in fig. 1.

In the second stage, for each specific task, we try to randomly compose the primitives into pipelines of warps, keep proposing instances with form $w_n(\cdots(w_j(w_i(x))))$. For each instance, we optimize all the constants used in its warps, $c_0, c_1, \cdots, c_n$, using gradient descent until convergence, by minimizing the KL divergence of pipeline PDF and target PDF. We keep track of the Pareto frontier of all the proposed instances with respect to complexity and minimum loss. It will be further discussed in section 3.2. Notice that this formulation does retain fig. 1 in the search space.

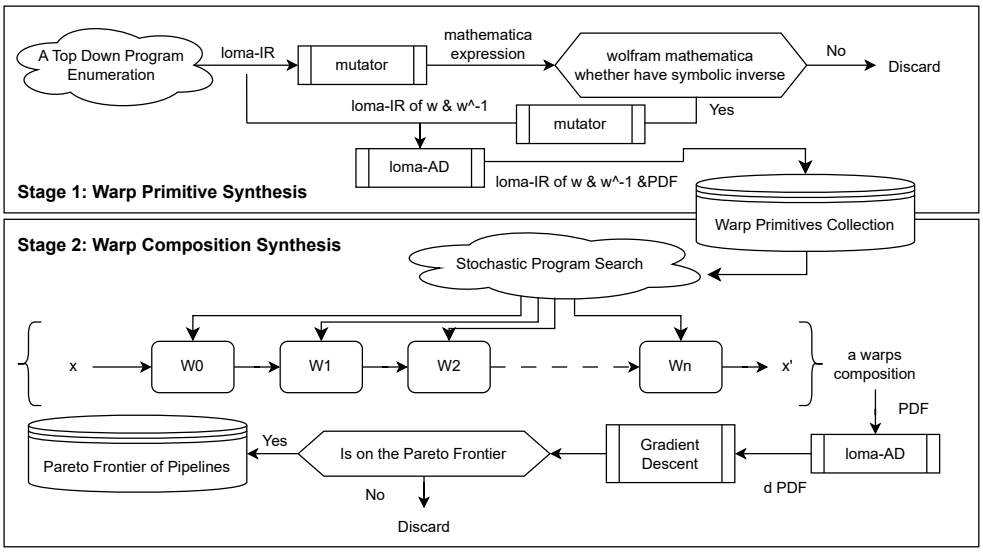

Fig. 2. The pipeline of our Symbolic Warp Synthesis. In stage 1, we try to enumerate as many simple warps as possible to ensure invertibility. In stage 2, we try to compose simple warps to enhance expressiveness while preserving invertibility. Essentially, we use two different program synthesisers for each stage.

3.1 Warp 1D Primitive Synthesis

Warp 1D Primitive. We use program synthesis to enumerate basic 1D warp primitives, which are necessary to be invertible. For efficiency concerns, we further constrain the warps to be symbolically invertible to avoid iterative solving inverse in runtime, e.g. using the bisection method. Directly enumerating all such expressions is extremely hard as far as we know, so we use reject sampling. Specifically, we first use a top-down explicit expression enumeration to enumerate all potential expressions and only collect those that have symbolic reverse.

Top Down Explicit Enumeration. To propose expressions, we use top-down explicit enumeration, which has proven to be particularly effective in constructing functional programs. Our synthesis is based on loma-IR and only uses the most basic functionalities:

```
expr = x | constant
     | [pow, sin, cos, sqrt, exp, log] expr
     | [+, -, *, /, pow] expr expr
```

Find Symbolic Inverse. Given an expression, it is very untrivial to find its symbolic inverse, so we resort to using Wolfram Mathematica [7]. To achieve this, we first use a mutator to compile loma-IR into a Mathematica expression, query its inverse via wolframclient API, and use another mutator to convert the inverse function back to loma-IR.

Equivalence Elimination. The number of expressions enumerated grows exponentially with respect to the depth. It will be important to eliminate the equivalence expressions, which can not only reduce the number of mathematia queries, but more importantly, also dramatically reduce the search space for stage 2. We heuristically classify the equivalence into several categories:

(1) *Constant Not Folding*: expressions like $[x + \cos(c_0 + c_1)]$ is not useful, as there is always an equivalence $[x + c_0]$ that has been enumerated before, with a smaller depth.
(2) *Early Composition*: expressions like $[\cos(x^2)]$ could be regarded as a composition of warp $[\cos(x)]$ and $[x^2]$, which will be enumerated in stage 2, so we should also cull these cases to prevent double counting.
(3) *Operand Exchange*: expressions like $[x + c_0]$ and $[c_0 + x]$ are just exchanging operands on both sides of a commutative operator, so only one of them should be left.
(4) *Generalization*: expressions like $[c_0x^2 + c_1x + c_2]$ is a generalization of all the specialized expressions like $[x^2 + c_0]$ or $[x^2 + c_1x]$, so ideally only the generalized version should be preserved. However, higher-order functions are always a generalization of lower-order ones; we probably want to preserve one generalization for each order.
(5) *Mathematically Equivalent*: expressions like $c_0[x - x]$ can actually be culled due to mathematical facts that all x terms just can cancel out. More complicated cases could be hard to detect with compiler, we probably would resort to manual work or mathematica.

Table 2. The number of expression synthesised, after culling, and invertible.

depth	explicit enum	constant folding	simple culling	invertible	equivalence	final
0	2	2	2	1	1	1
1	30	20	20	15	13	9
2	750	500	200	65	35	33
3	23250	14500	7000	3560	1498	417
4	806250	462500	245000	TBD	TBD	TBD

Our implementation was not able to eliminate all the equivalence listed above, but a subset of them is already enough for our next stage. We build our synthesis pipeline as multiple passes, each pass act like a filter to remove or mutate current expressions. During each pass, we save a copy of current filtered expressions so that they can be reused if filtering algorithm changed. The passes are listed below:

(1) Enumerate Pass: In this pass, we explictly enumerate all possible expressions with given depth to a large dictionary. We observed that functions such as `sqrt` and `/` can be considered as a specialized constant power, which can already be generated by the power function. Therefore, we have decided to exclude them in this stage.
(2) Constant Folding Pass: In this pass, we filter out expressions that have constants at the top level. Additionally, we remove expressions that contain sub-expressions which can be simplified to a single constant node.

(3) In this pass, we convert our Loma-IR expressions into Mathematica expressions and use Mathematica to find the inverse of the functions. Some functions may take a long time to invert, so we set a time limit of 10 seconds for finding the inversion. We use the Mathematica instruction `TimeConstrained[inverseSolution = Solve[y==f[x], x], 10]` for this purpose.
(4) Symbolic Equivalence Pass: In this pass, we will test each pair of expressions for symbolic equivalence. We use Mathematica instruction `SameQ[a, b]` for symbolic equivalence test.
(5) Redundant Expression Pass: In this pass, we remove the expressions although not symbolic equivalent, but can be a specialization of a previously generated expression. Such as $x+(x+x)$, $x\times(x-x)$, and $(x^{c0})^{c1}$. Where the operators on x can be converted to constant multiplication of x.
(6) Composition Test Pass: In this pass, we remove expressions that can be represented as the composition of two previously generated expressions. We maintain a list of generated expressions from previous depths. For each pair of previous expressions, we compose them and compare the result with expressions of the current depth. Then we use Mathematica to test for symbolic equivalence between the composed expressions and the current expressions.

We have basically set up the system, and some basic statistics are shown in table 2.

3.2 RAWS: Range Aware Warp Synthesizer

In this section, we present yet another warp synthesizer, RAWS, that permutates and composes all the primitives generated in the previous stage, to fit a specific target PDF.

3.2.1 Range Aware Warp Primitives. Given a symbolic warp synthesized in the previous stage, its invertibility could be heavily related to the input domain. For example, **sin(x)** is only invertible between $[-\frac{\pi}{2}, +\frac{\pi}{2}]$, considering the actual range of its inverse **arcsin(z)**. To make things worse, a warp's input domain is determined by the previous warp's output range, and the range could be very different. For example, **sin(x)** could give anything between $[-1, 1]$, while **tan(x)** can give any real number. Therefore, various warps' composition needs to be cautiously approached; otherwise, NaN would be everywhere due to being out of domain and range.

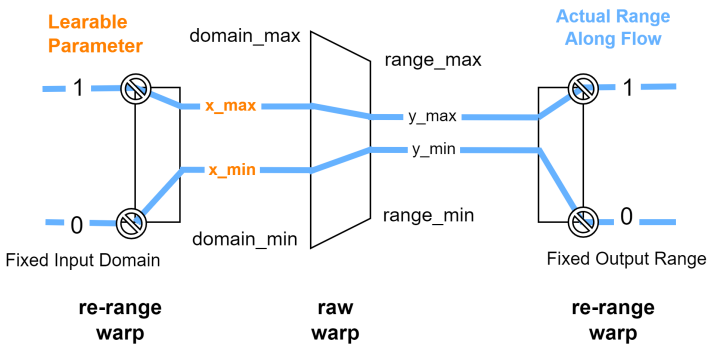

Fig. 3. Our re-range sandwich wrapper on a raw symbolic warp. As a whole, it always takes input in the domain $[0, 1]$; then the first re-range warp maps the input to a learnable range $[x_{min}, x_{max}]$, which is guaranteed to be within the valid domain of the raw warp, i.e. $[x_{min}, x_{max}] \subseteq [domain_{min}, domain_{max}]$. The constraint is solved by applying projected gradient descent. Later, the output of the raw warp is always mapped back to $[0, 1]$ by the second re-range warp.

To resolve the issue, we force every primitive to have a deterministic input domain $[0, 1]$ and output range $[0, 1]$. If a primitive does not meet the requirement, we always sandwich it with two re-range warps that can map input in $[0, 1]$ to a reasonable range for the raw primitive and later scale the output back to $[0, 1]$, as described in fig. 3.

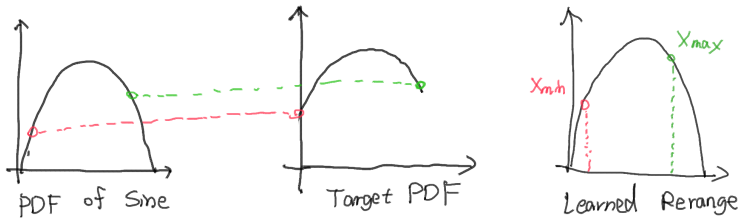

Fig. 4. The PDF of a sine warp has a simple concave shape. If we do not use any re-range, this primitive cannot provide any freedom to finetune the warp. The sine warp itself cannot even fit the curve of a truncated sine. However, adding a re-range sandwich could perfectly present the truncated sine with only two more affine transform, while both the domain and range are well preserved to $[0, 1]$.

To learn a reasonable input range, The sandwich re-range wrapper introduces 2 parameters, x_{min} and x_{max}. These parameters are very useful, as they enable us to use any curve segment of the PDF curve rather than having to use the entire curve. This greatly enhances the representability. fig. 4 shows an example that by sandwiching a raw warp, we can perfectly fit some PDF curves that the raw warp itself cannot.

Meanwhile, this design helps us to do mutation and hybridization during the evolutionary search, as the entire flow can be arbitrarily modified without causing any range-domain mismatching between neighbor primitives.

3.2.2 Evolutionary Algorithm for Flow Synthesis. To enhance the complexity of the symbolic warps, we further synthesize more involved models by compositing all the primitives enumerated[1]:

```
warp-flow := warp(x)
           | warp(warp-flow(x))
```

Different target PDFs may need different permutations and compositions, and we use the evolutionary algorithm to find the best-matching warp flow, as depicted in fig. 5.

3.2.3 Mixture of Warps. In scenarios where the target distribution is multimodal, fitting a single warp flow can be challenging due to the complexity of the distribution. This is due to multimodal functions having their CDFs change the increasing speed multiple times; however, our primitive functions mostly have a smooth change of increasing rate. We propose to use the Mixture of Warps (MoW), which combines multiple warp flows to address this issue. By combining the strengths of the mixture models and warp flows, our method aims to capture the intricate details of the target distribution more effectively.

Specifically, we train a mixture of warp flows, where each component in the mix is modelled by a distinct warp flow. This allows each flow to specialize in a particular mode or peak of the distribution, enhancing the overall fitting capability. During training, the Expectation-Maximization (EM) algorithm is employed to iteratively update the parameters of the mixture components, ensuring that each warp flow learns to represent its assigned portion of the data accurately.

[1]At this stage, we only use a very limited set of warps for simplicity.

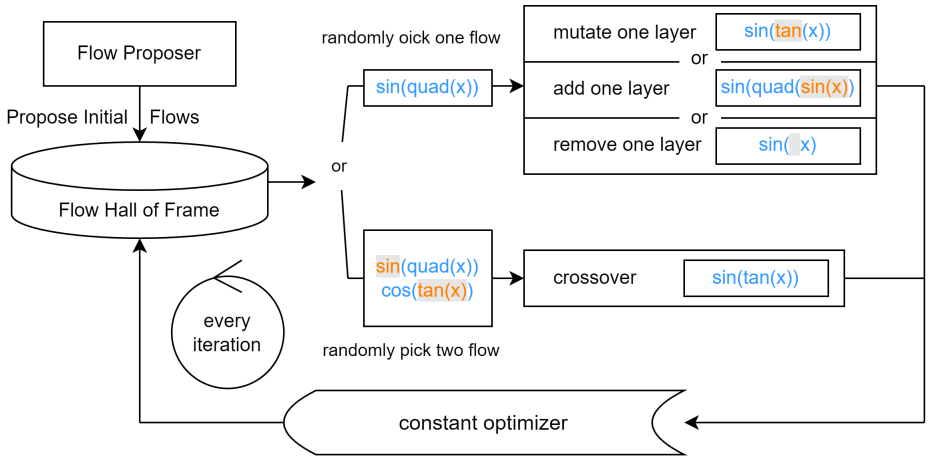

Fig. 5. We use an evolutionary algorithm to search the optimal composition of symbolic flow. (1) It starts with randomly composing a few warp flows and then storing them in a pool. (2) Then, we randomly pick one or two flows from the pool in every adaptation iteration and make some mutations and hybridizations. (3) After that, we use a gradient descent optimizer to optimize the parameter vector for the new warp flow. (4) Finally, we push the new flow to the pool, sort all flows with some criteria and only preserve the TOP 5,

The warp flow can be arbitrary functions that satisfy the constraints described before. To simplify the setting, in validation we only focus on fitting the truncated GMM model for now. Therefore, we designed a sigmoid warp function that could approximate the Gaussian distribution. The formula for this function is written as:

$$f(x) = \frac{1}{\exp(-a(x-0.5)+1}$$

where a controls how thin or fat the peak component would be (like the σ of the Gaussian Distribution), and we combine the function with re-range sandwich described before, so that we can also learn to decide the relative position of the peak (like the μ of the Gaussian Distribution).

4 WARP 2D SYNTHESIS

The high-level idea is first to marginalize the target distribution over y and fit a 1D warp on $p(x)$. Then, we try to find another optimal 1D warp expression that fits all $p(y|x)$ well. Specifically, we sample many x_i points, and for each proposed instance, we optimize it for every $p(y|x_i)$ respectively and compute the sum of minimum KL-divergence. Once we find the optimal warp for $p(y|x)$, we can use conventional symbolic regression to fit $c_0, c_1, \cdots, c_n = f(x)$, as it is unconstrained. A more detailed pipeline is shown in section 5.2.

5 EVALUATION

5.1 Warp Synthesis for 1D PDF

1D PDF Fitting Comparison. We use our RAWS system to fit various simple 1D toy PDFs, as shown in fig. 6. The target PDF shape is constructed by truncating Gaussian, sine and other simple functions, so there is no analytical way to sample from them. We compare our approach with the piecewise-quadratic spline warp [9] without MLP encoding.

Our approach can always produce a smoother PDF curve than the piecewise-quadratic spline, as the latter's PDF is eventually a piecewise-linear function. Furthermore, in some cases, our approach

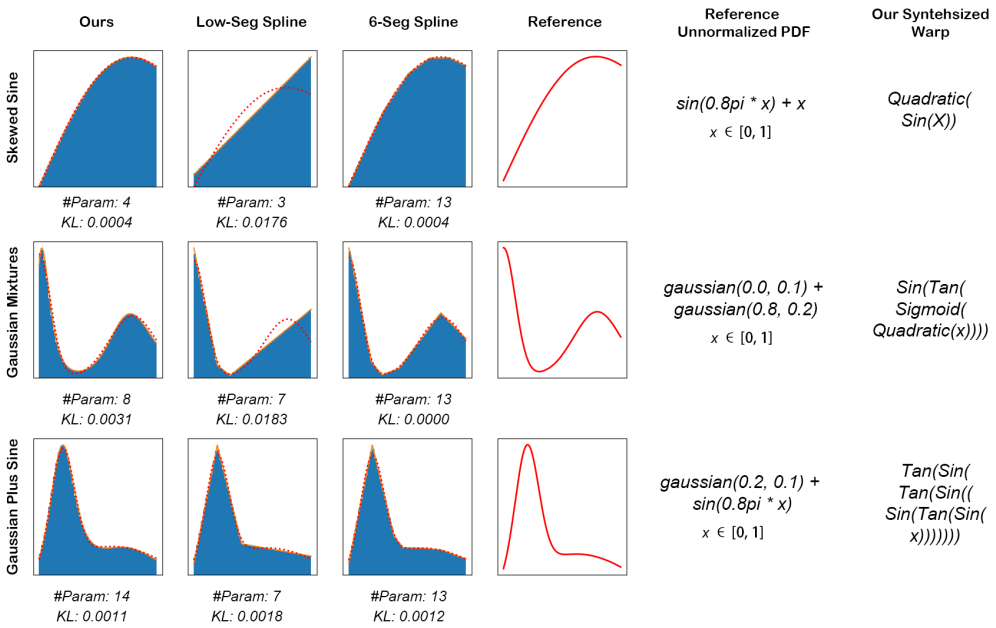

Fig. 6. We compare our synthesised warp with a general 1D piecewise-quadratic spline warp [9], with both lower and relatively higher (6) number of segmentations.

could achieve a better fit with a lower number of parameters. Intuitively, this is because our model has a specialized thus shrank hypothesis space, which needs less effort to describe the optimal point.

Unfortunately, our primitive set is still minimal, and therefore, we still need a relatively large number of parameters for more involved cases. For example, in the Gaussian Plus Sine case, we synthesised a flow with 7 layers to achieve a reasonable fit. If we have a very strong layer that can fairly fit the shape by itself, we will have far less depth and, thus, far fewer parameters.

A fun deep learning analogy is the width and depth of the multi-layer perceptron (MLP). Using more involved primitives is like creating a wider hidden layer, and piling more primitives is like increasing the number of hidden layers.

Intrinsic Semantic of Synthesised Warps. Once we have found a specific symbolic flow, we observe that its parameter vector could have meaningful semantics for the curve shape of the PDF, as depicted in fig. 7. This feature implies our parameters are far more interpretable than those of a neural network or a piecewise spline.

Furthermore, it reveals our parameter space spans a specific family of curves for PDFs; in this case, it is the ones with two Gaussian-like lobes. Despite it somehow means we have a smaller space to optimize, this is beneficial if we do precisely want this family of curves because our model will need fewer parameters and computation as it embeds super strong prior.

1D PDF Fitting with Mixture of Flows. We used a mixture of flows to fit several multimodal distributions and found it could accurately sample the distribution while using fewer parameters compared to traditional methods such as the piecewise-quadratic spline warp method. In our experiment, we use a truncated mixture of Gaussians to represent the target PDF function; each target distribution is designed to have multiple peaks with varying heights and widths. The results

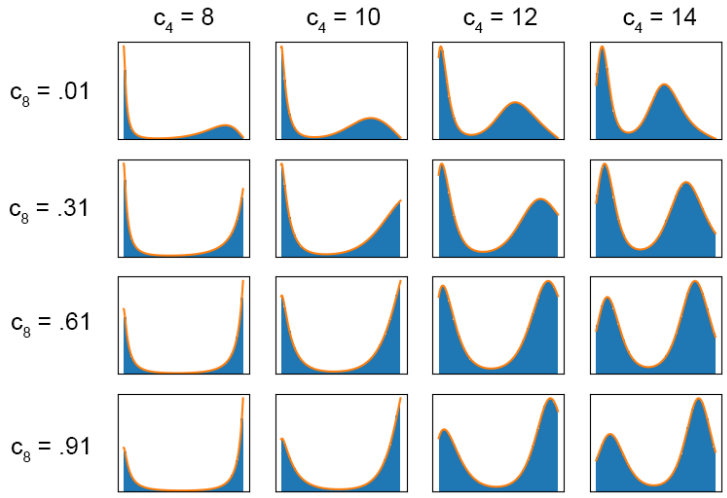

Fig. 7. Given a synthesised flow, we could manually tune its parameter vector. We observe that the constant parameters can imply some meaningful features for the curve shape of the PDF: here, c_4 determines weights of the RHS lobe, and c_8 is more about shifting the mean of the RHS lobe.

are shown in fig. 8, where under the same number of parameters, our mixture of flows achieves lower KL divergence compared to the piecewise-quadratic spline warp method.

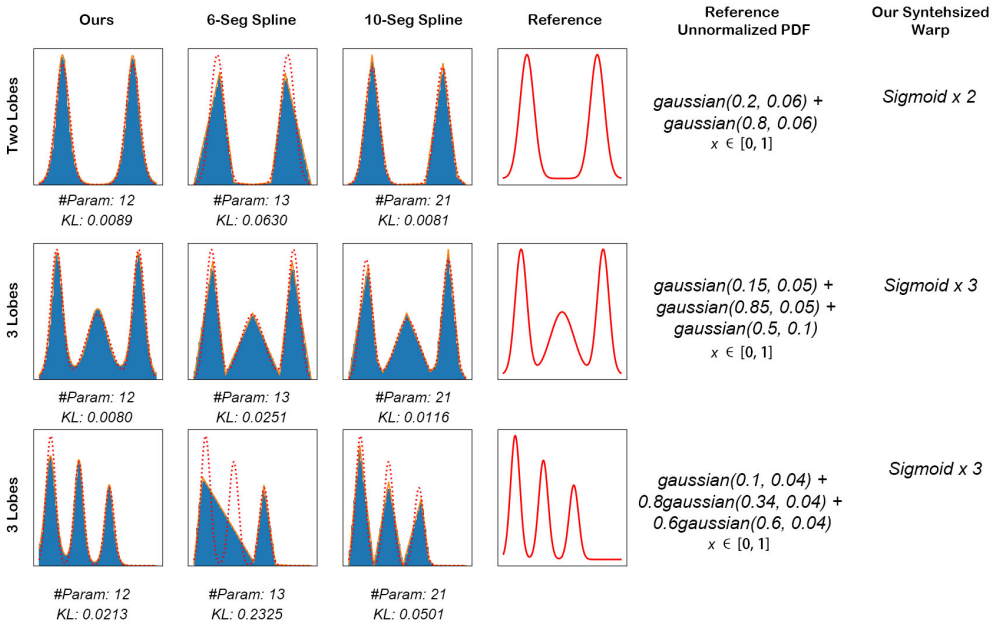

Fig. 8. We compare our mixture of warps method on Gaussian mixture distributions with 1D piecewise-quadratic spline warp [9].

5.2 Warp Synthesis for 2D PDF

We also showcased RAWS's ability to fit 2D PDFs, as depicted in fig. 9.

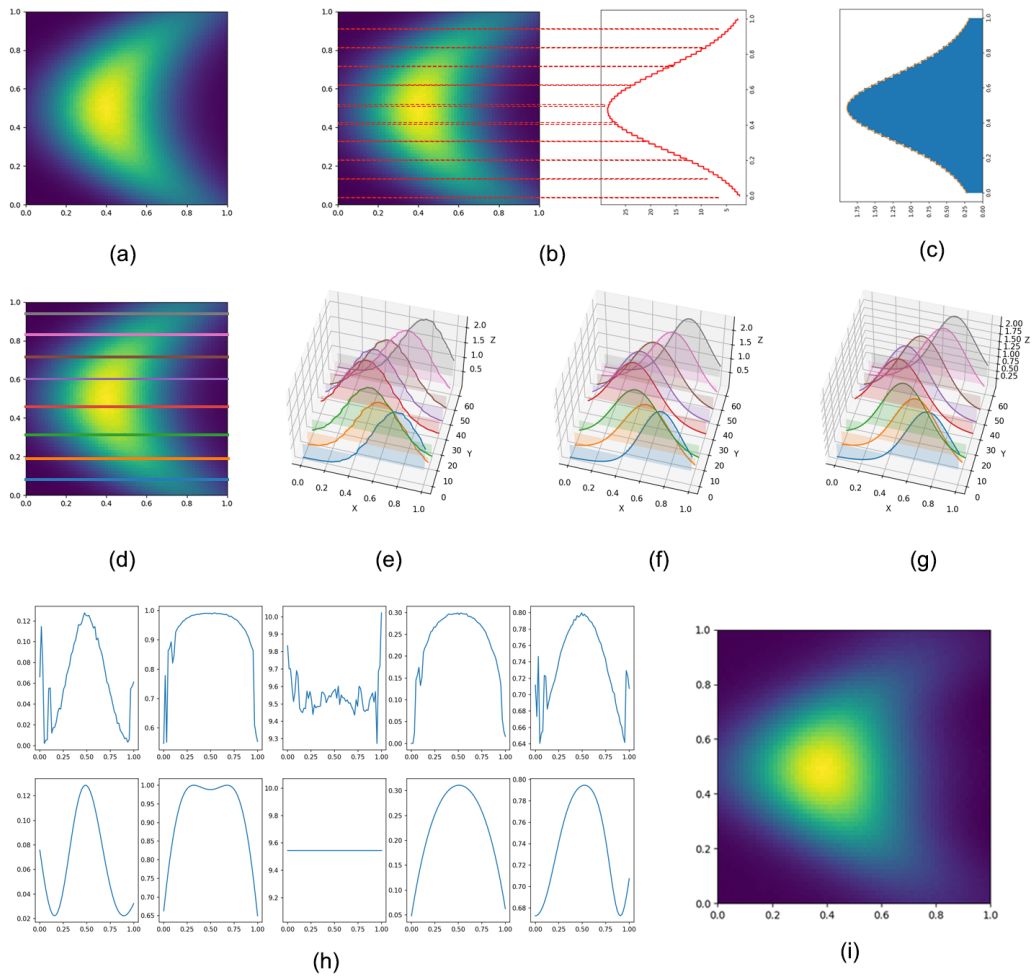

Fig. 9. (a) Given a target 2D distribution in Primal Sample Space (PSS), which is a truncated Banana in this case, without analytically sampling routine. (b) We first marginalize the distribution over the x-axis to get a 1D target PDF for y-axis, and (c) learn a symbolic warp to approximate it. (d) Then, we need to learn the x-axis warp, which is conditional on Y. Here, we show (e) a few slices of these target conditional x-warps. (f) We use another symbolic warp to approximate this, which is **sigmoid(sin(x))** here, where the parameters of the symbolic warps are learned individually for each Y value. (h) The x-axis symbolic warp has 5 parameters, which are eventually 5 curves mapping Y value to the target parameters, and we use a general symbolic regression framework [2] to approximate the learned parameters (first row) with symbolic functions (second row). Putting everything together, we can achieve a fully symbolical sampling routine for the 2D distribution, with sliced x-warps in (g) and final sample density in (i).

6 FUTURE WORK

More Primitives. Now, we only use a minimal set of primitives with handcraft. Eventually, we would like to use all the primitives synthesized in stage 1. This poses challenges on how to determine the domain & range automatically and needs another code generator to compile LOMA to Python "Primitive" class.

Pareto Frontier for Practical Trade-Off. Theoretically, our synthesizer is able to explore a Pareto Frontier for efficiency-accuracy trade-off. However, it requires us to evaluate the efficiency of each symbolic warp primitive, which needs a closer investigation, so we leave it as future work.

Piecewise-Warp Model. Besides the Mixture of Warps (MoW), a piecewise-warp model is another choice to handle more involved cases. We allocate cutpoints on the domain and use a different warp for each segment of the domain. The challenges include discontinuity-aware gradient computation and automatically determining the optimal number of cutpoints.

Generalized Flow Synthesiser. Our flow synthesizer is only able to synthesize simple compositions of primitives now. Eventually, we would like to includes Mixture of Flows, Piecewise-Warp Flows, Piecewise-Warp of Mixtures, Mixture of Mixture, Mixture of piece-wise-warp etc.

Effective Ways to Train the Model. Gradient descent optimization for complex, nested warps can be challenging. The optimization landscape may be highly non-convex, with many local minima, making it difficult to find the global optimum. This problem is particularly prominent in the MoW method, the performance of the optimization can be highly sensitive to the initial values of the parameters and the learning rate when there are multiple lobes.

REFERENCES

[1] James Arvo. 1995. Stratified sampling of spherical triangles. In *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '95)*. Association for Computing Machinery, New York, NY, USA, 437–438. https://doi.org/10.1145/218380.218500

[2] Miles Cranmer. 2023. Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl. arXiv:2305.01582

[3] Stavros Diolatzis, Adrien Gruson, Wenzel Jakob, Derek Nowrouzezahrai, and George Drettakis. 2020. Practical Product Path Guiding Using Linearly Transformed Cosines. *In Computer Graphics Forum (Proceedings of Eurographics Symposium on Rendering)* 39, 4 (July 2020).

[4] Ana Dodik, Marios Papas, Cengiz Öztireli, and Thomas Müller. 2022. Path Guiding Using Spatio-Directional Mixture Models. In *Computer Graphics Forum*, Vol. 41. Wiley Online Library, 172–189.

[5] David Hart, Matt Pharr, Thomas Müller, Ward Lopes, Morgan McGuire, and Peter Shirley. 2020. Practical product sampling by fitting and composing warps. In *Computer Graphics Forum*, Vol. 39. Wiley Online Library, 149–158.

[6] Eric Heitz. 2018. Sampling the GGX Distribution of Visible Normals. *Journal of Computer Graphics Techniques (JCGT)* 7, 4 (30 November 2018), 1–13. http://jcgt.org/published/0007/04/01/

[7] Wolfram Research, Inc. [n. d.]. Mathematica, Version 14.0. https://www.wolfram.com/mathematica Champaign, IL, 2024.

[8] Christopher Lutz and Howard Derby. 1986. Janus: a time-reversible language. *Letter to R. Landauer* 2 (1986).

[9] Thomas Müller, Brian McWilliams, Fabrice Rousselle, Markus Gross, and Jan Novák. 2019. Neural Importance Sampling. *ACM Trans. Graph.* 38, 5, Article 145 (Oct. 2019), 19 pages. https://doi.org/10.1145/3341156

[10] Carlos Ure na and Iliyan Georgiev. 2018. Stratified Sampling of Projected Spherical Caps. *Computer Graphics Forum (Proceedings of EGSR)* 37, 4 (2018).

[11] George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. 2021. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research* 22, 57 (2021), 1–64.

[12] Christoph Peters and Carsten Dachsbacher. 2019. Sampling Projected Spherical Caps in Real Time. *Proc. ACM Comput. Graph. Interact. Tech.* 2, 1 (2019). https://doi.org/10.1145/3320282

[13] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Xiaojiang Chen, and Xin Wang. 2021. A comprehensive survey of neural architecture search: Challenges and solutions. *ACM Computing Surveys (CSUR)* 54, 4 (2021), 1–34.

[14] Carlos Ureña, Marcos Fajardo, and Alan King. 2013. An area-preserving parametrization for spherical rectangles. In *Proceedings of the Eurographics Symposium on Rendering* (Zaragoza, Spain) *(EGSR '13)*. Eurographics Association, Goslar, DEU, 59–66. https://doi.org/10.1111/cgf.12151

[15] Jiří Vorba, Johannes Hanika, Sebastian Herholz, Thomas Müller, Jaroslav Křivánek, and Alexander Keller. 2019. Path Guiding in Production. In *ACM SIGGRAPH 2019 Courses* (Los Angeles, California) *(SIGGRAPH '19)*. ACM, New York, NY, USA, Article 18, 77 pages. https://doi.org/10.1145/3305366.3328091